

HR Employee Attrition Analysis

Data Analysis & Machine Learning Pipeline

```
from google.colab import files
uploaded =files.upload()
```

Choose files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
#Import some basic libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

Data Analysis

```
# Read the dataset
df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')

# Preview top 5 rows
df.head(5)
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNum
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	

5 rows × 35 columns

```
# Last 10 rows
df.tail(10)
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber
1460	29	No	Travel_Rarely	468	Research & Development	28	4	Medical	1	1460
1461	50	Yes	Travel_Rarely	410	Sales	28	3	Marketing	1	1461
1462	39	No	Travel_Rarely	722	Sales	24	1	Marketing	1	1462
1463	31	No	Non-Travel	325	Research & Development	5	3	Medical	1	1463
1464	26	No	Travel_Rarely	1167	Sales	5	3	Other	1	1464
1465	36	No	Travel_Frequently	884	Research & Development	23	2	Medical	1	1465
1466	39	No	Travel_Rarely	613	Research & Development	6	1	Medical	1	1466
1467	27	No	Travel_Rarely	155	Research & Development	4	3	Life Sciences	1	1467
1468	49	No	Travel_Frequently	1023	Sales	2	3	Medical	1	1468
1469	34	No	Travel_Rarely	628	Research & Development	8	3	Medical	1	1469

10 rows × 35 columns

```
# Dataset shape (rows, columns)
df.shape
```

```
(1470, 35)
```

```
# Data types and non-null counts
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    1470 non-null   int64
1   Attrition                            1470 non-null   object
2   BusinessTravel                        1470 non-null   object
3   DailyRate                            1470 non-null   int64
4   Department                           1470 non-null   object
5   DistanceFromHome                     1470 non-null   int64
6   Education                             1470 non-null   int64
7   EducationField                        1470 non-null   object
8   EmployeeCount                         1470 non-null   int64
9   EmployeeNumber                       1470 non-null   int64
10  EnvironmentSatisfaction               1470 non-null   int64
11  Gender                               1470 non-null   object
12  HourlyRate                           1470 non-null   int64
13  JobInvolvement                       1470 non-null   int64
14  JobLevel                             1470 non-null   int64
15  JobRole                              1470 non-null   object
16  JobSatisfaction                       1470 non-null   int64
17  MaritalStatus                        1470 non-null   object
18  MonthlyIncome                        1470 non-null   int64
19  MonthlyRate                          1470 non-null   int64
20  NumCompaniesWorked                   1470 non-null   int64
21  Over18                               1470 non-null   object
22  OverTime                             1470 non-null   object
23  PercentSalaryHike                    1470 non-null   int64
24  PerformanceRating                    1470 non-null   int64
25  RelationshipSatisfaction              1470 non-null   int64
26  StandardHours                        1470 non-null   int64
27  StockOptionLevel                     1470 non-null   int64
28  TotalWorkingYears                    1470 non-null   int64
29  TrainingTimesLastYear                1470 non-null   int64
30  WorkLifeBalance                      1470 non-null   int64
31  YearsAtCompany                       1470 non-null   int64
32  YearsInCurrentRole                   1470 non-null   int64
33  YearsSinceLastPromotion              1470 non-null   int64
34  YearsWithCurrManager                 1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

```
# Check missing values
df.isnull().sum()
```

	0
Age	0
Attrition	0
BusinessTravel	0
DailyRate	0
Department	0
DistanceFromHome	0
Education	0
EducationField	0
EmployeeCount	0
EmployeeNumber	0
EnvironmentSatisfaction	0
Gender	0
HourlyRate	0
JobInvolvement	0
JobLevel	0
JobRole	0
JobSatisfaction	0
MaritalStatus	0
MonthlyIncome	0
MonthlyRate	0
NumCompaniesWorked	0
Over18	0
OverTime	0
PercentSalaryHike	0
PerformanceRating	0
RelationshipSatisfaction	0
StandardHours	0
StockOptionLevel	0
TotalWorkingYears	0
TrainingTimesLastYear	0
WorkLifeBalance	0
YearsAtCompany	0
YearsInCurrentRole	0
YearsSinceLastPromotion	0
YearsWithCurrManager	0

dtype: int64

```
# Statistical summary
df.describe()
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRat
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	1470.000000	1470.000000	1470.00000
mean	36.923810	802.485714	9.192517	2.912925	1.0	1024.865306	2.721769	65.89115
std	9.135373	403.509100	8.106864	1.024165	0.0	602.024335	1.093082	20.32942
min	18.000000	102.000000	1.000000	1.000000	1.0	1.000000	1.000000	30.00000
25%	30.000000	465.000000	2.000000	2.000000	1.0	491.250000	2.000000	48.00000
50%	36.000000	802.000000	7.000000	3.000000	1.0	1020.500000	3.000000	66.00000
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1555.750000	4.000000	83.75000
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.000000	4.000000	100.00000

8 rows × 26 columns

```
# Column data types
df.dtypes
```

	0
Age	int64
Attrition	object
BusinessTravel	object
DailyRate	int64
Department	object
DistanceFromHome	int64
Education	int64
EducationField	object
EmployeeCount	int64
EmployeeNumber	int64
EnvironmentSatisfaction	int64
Gender	object
HourlyRate	int64
JobInvolvement	int64
JobLevel	int64
JobRole	object
JobSatisfaction	int64
MaritalStatus	object
MonthlyIncome	int64
MonthlyRate	int64
NumCompaniesWorked	int64
Over18	object
OverTime	object
PercentSalaryHike	int64
PerformanceRating	int64
RelationshipSatisfaction	int64
StandardHours	int64
StockOptionLevel	int64
TotalWorkingYears	int64
TrainingTimesLastYear	int64
WorkLifeBalance	int64
YearsAtCompany	int64
YearsInCurrentRole	int64
YearsSinceLastPromotion	int64
YearsWithCurrManager	int64

dtype: object

All column names
df.columns

```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',  
      'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',  
      'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',  
      'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',  
      'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',  
      'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',  
      'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',  
      'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',  
      'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',  
      'YearsWithCurrManager'],  
      dtype='object')
```

```
# Value counts for Department
df['Department'].value_counts()
```

	count
Department	
Research & Development	961
Sales	446
Human Resources	63

dtype: int64

```
# Value counts for JobRole
df['JobRole'].value_counts()
```

	count
JobRole	
Sales Executive	326
Research Scientist	292
Laboratory Technician	259
Manufacturing Director	145
Healthcare Representative	131
Manager	102
Sales Representative	83
Research Director	80
Human Resources	52

dtype: int64

```
# Value counts for Attrition (target variable)
df['Attrition'].value_counts()
```

	count
Attrition	
No	1233
Yes	237

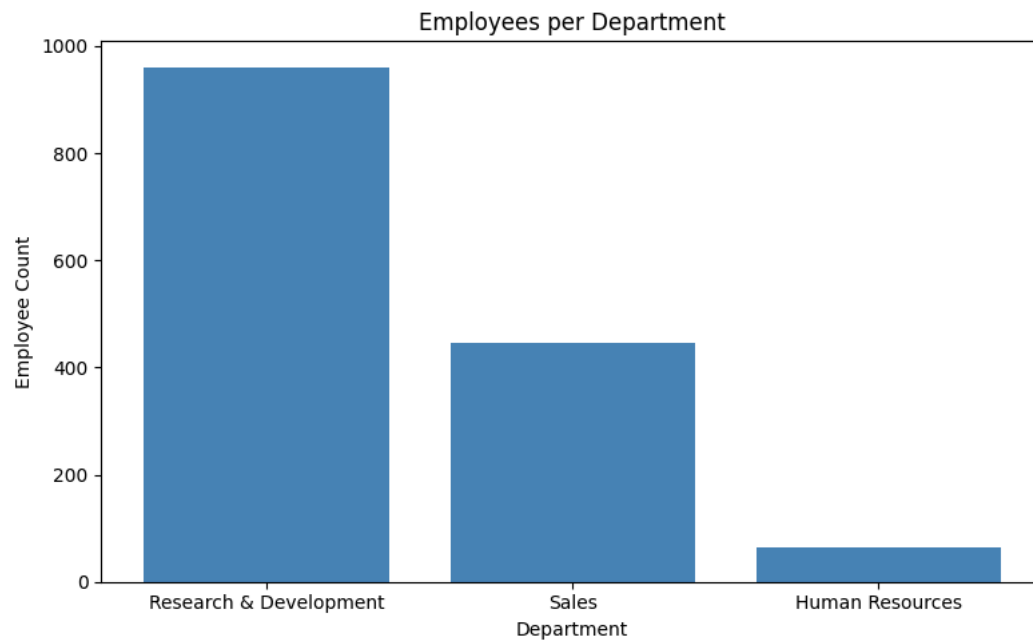
dtype: int64

```
# Drop constant/irrelevant columns
df.drop(columns=['EmployeeCount', 'Over18', 'StandardHours'], inplace=True)
print('Dropped constant columns. Remaining shape:', df.shape)
```

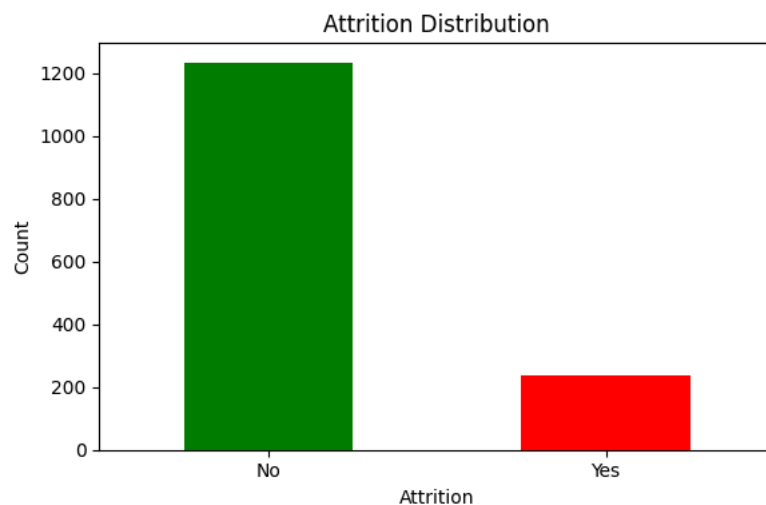
Dropped constant columns. Remaining shape: (1470, 32)

▼ Data Visualization

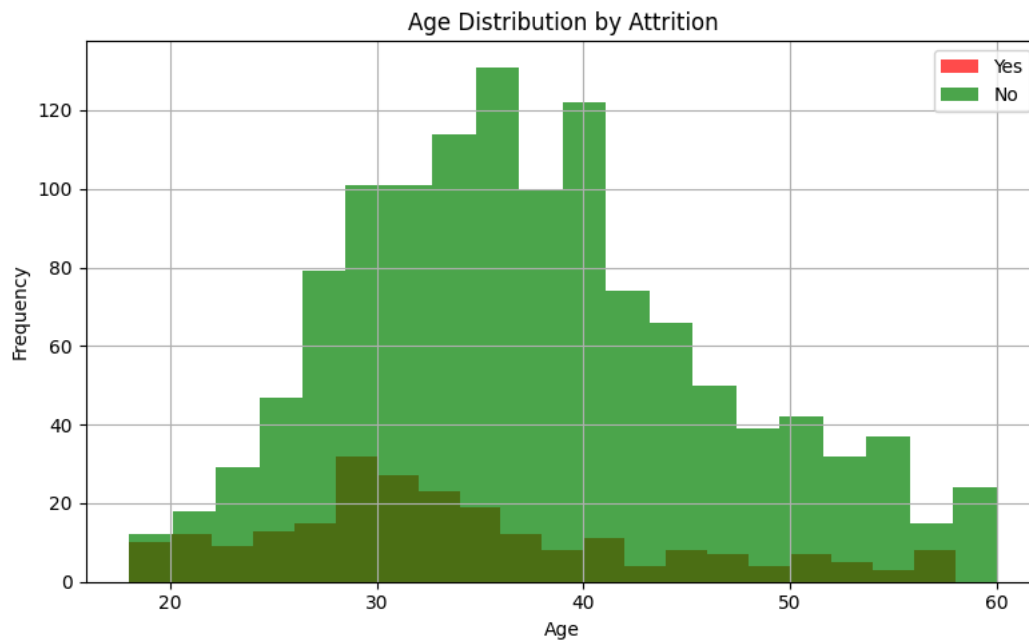
```
# 1. Attrition count by Department
plt.figure(figsize=(8, 5))
plt.bar(df['Department'].value_counts().index,
        df['Department'].value_counts().values, color='steelblue')
plt.xlabel('Department')
plt.ylabel('Employee Count')
plt.title('Employees per Department')
plt.tight_layout()
plt.show()
```



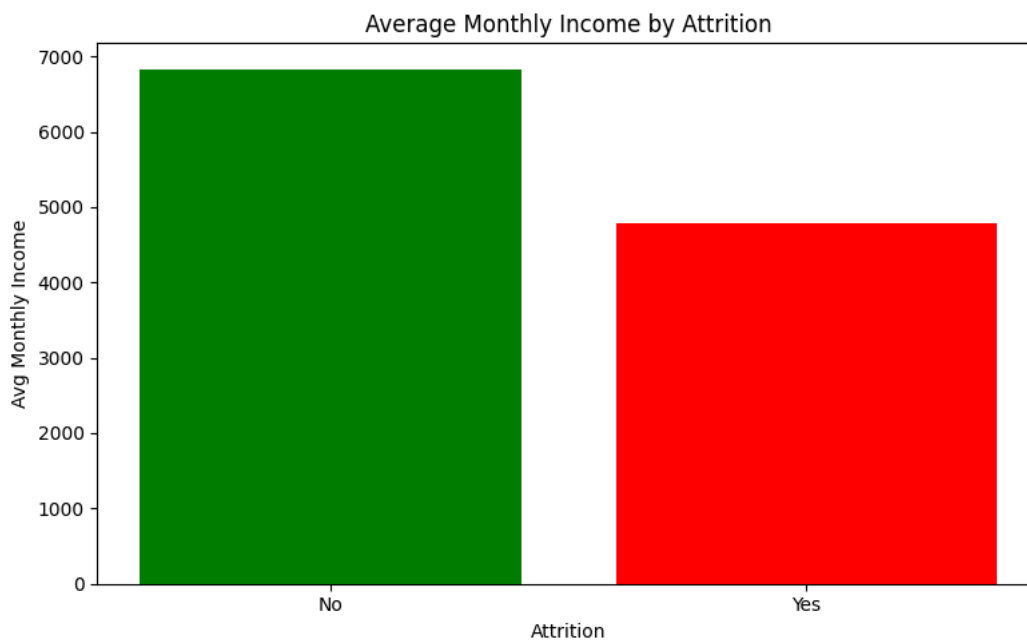
```
# 2. Attrition distribution (Yes vs No)
plt.figure(figsize=(6, 4))
df['Attrition'].value_counts().plot(kind='bar', color=['green', 'red'])
plt.xlabel('Attrition')
plt.ylabel('Count')
plt.title('Attrition Distribution')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



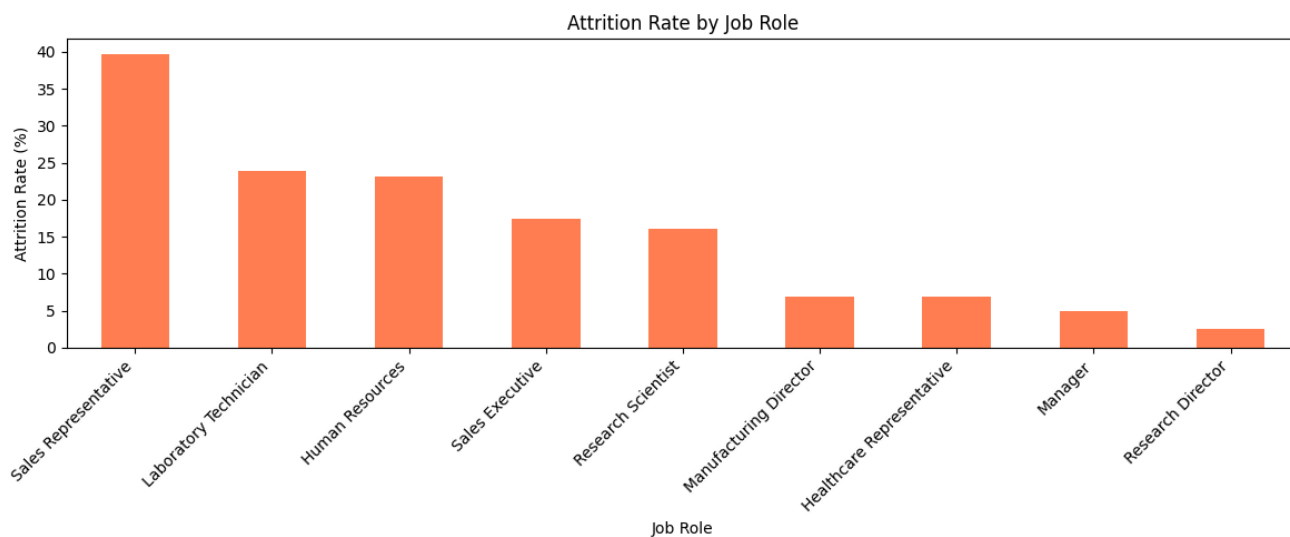
```
# 3. Age distribution by Attrition
plt.figure(figsize=(8, 5))
df[df['Attrition'] == 'Yes']['Age'].hist(alpha=0.7, label='Yes', color='red', bins=20)
df[df['Attrition'] == 'No']['Age'].hist(alpha=0.7, label='No', color='green', bins=20)
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Distribution by Attrition')
plt.legend()
plt.tight_layout()
plt.show()
```



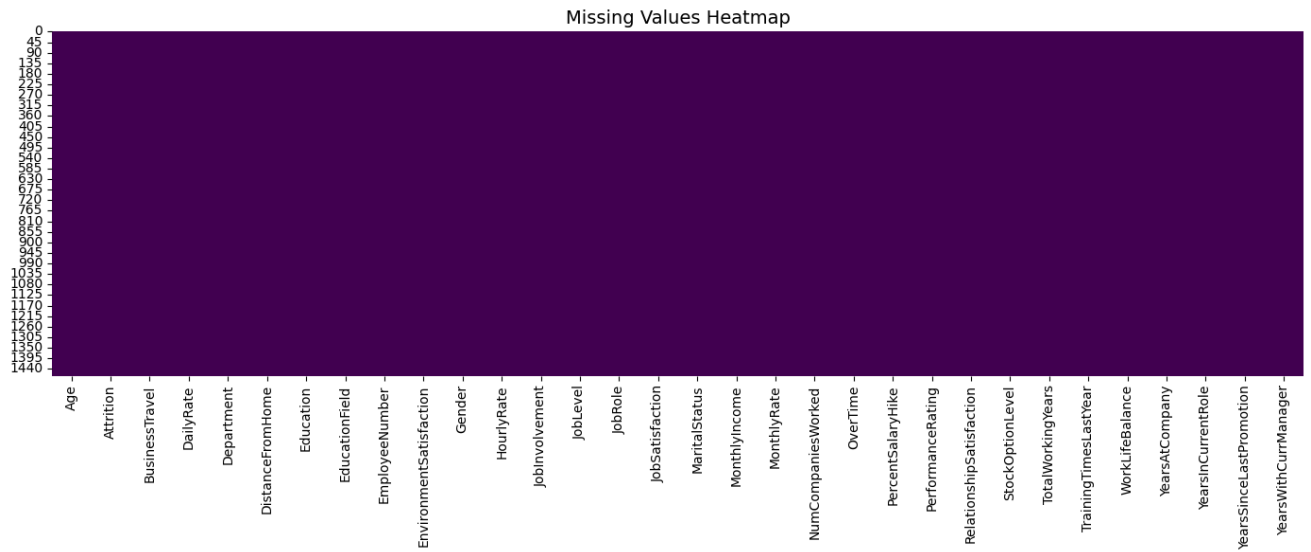
```
# 4. MonthlyIncome vs Attrition (bar chart)
plt.figure(figsize=(8, 5))
avg_income = df.groupby('Attrition')['MonthlyIncome'].mean()
plt.bar(avg_income.index, avg_income.values, color=['green', 'red'])
plt.xlabel('Attrition')
plt.ylabel('Avg Monthly Income')
plt.title('Average Monthly Income by Attrition')
plt.tight_layout()
plt.show()
```



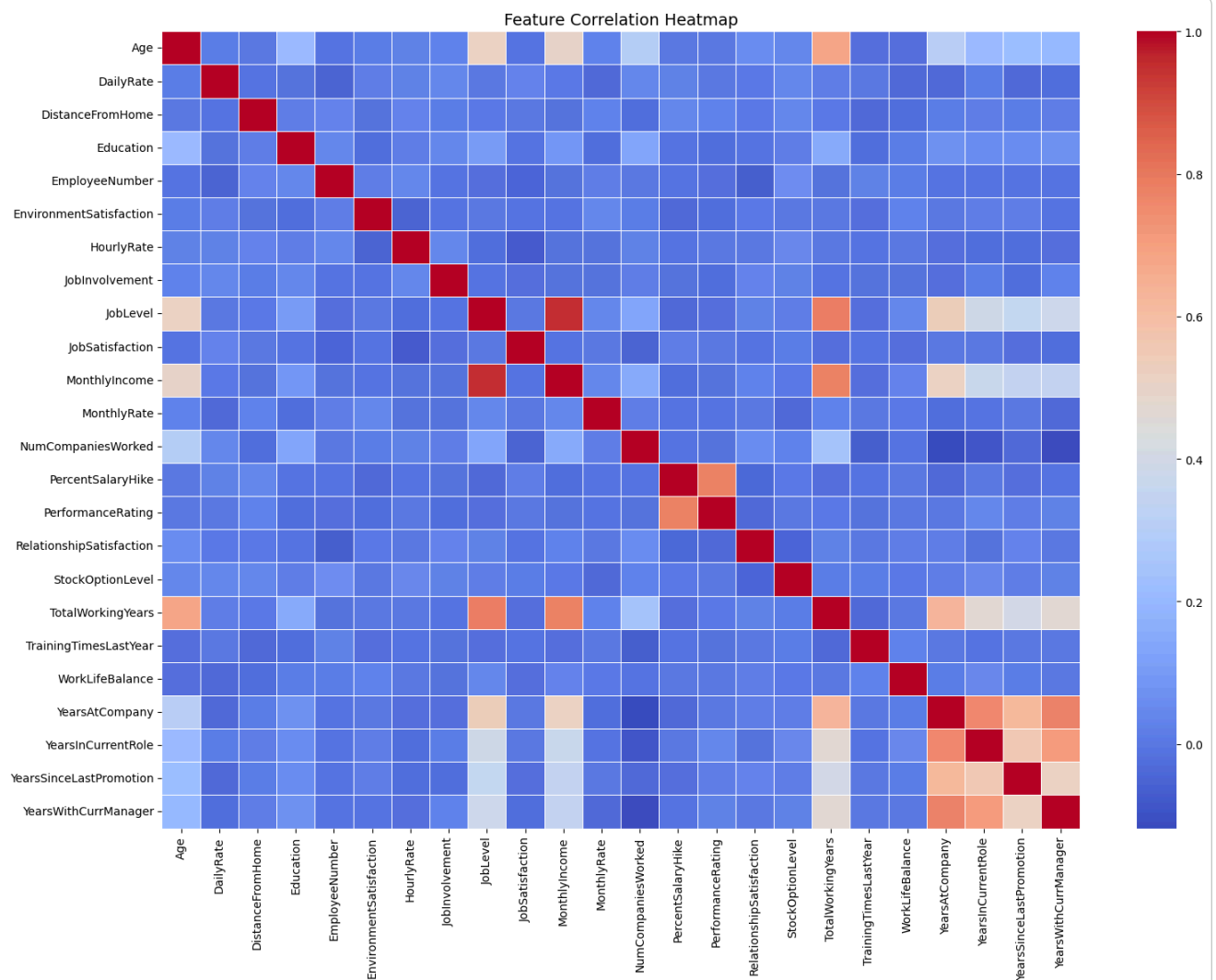
```
# 5. Attrition rate by JobRole
plt.figure(figsize=(12, 5))
jobrole_attrition = df.groupby('JobRole')['Attrition'].apply(
    lambda x: (x == 'Yes').sum() / len(x) * 100
).sort_values(ascending=False)
jobrole_attrition.plot(kind='bar', color='coral')
plt.xlabel('Job Role')
plt.ylabel('Attrition Rate (%)')
plt.title('Attrition Rate by Job Role')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



```
# 6. Missing Values Heatmap
plt.figure(figsize=(14, 6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap', fontsize=14)
plt.tight_layout()
plt.show()
```



```
# 7. Correlation heatmap (numeric columns only)
plt.figure(figsize=(16, 12))
numeric_df = df.select_dtypes(include=[np.number])
sns.heatmap(numeric_df.corr(), annot=False, cmap='coolwarm', linewidths=0.5)
plt.title('Feature Correlation Heatmap', fontsize=14)
plt.tight_layout()
plt.show()
```



Machine Learning

Preprocessing & Feature Engineering

```
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer

# Encode all categorical columns
df_ml = df.copy()
le = LabelEncoder()
cat_cols = df_ml.select_dtypes(include='object').columns
for col in cat_cols:
    df_ml[col] = le.fit_transform(df_ml[col])
```

```
print('Categorical columns encoded:', list(cat_cols))
```

```
Categorical columns encoded: ['Attrition', 'BusinessTravel', 'Department', 'EducationField', 'Gender', 'JobRole', 'MaritalStatus']
```

```
# DISTRIBUTE X (features) and y (target)
feature_cols = [col for col in df_ml.columns if col != 'Attrition']
X = df_ml[feature_cols].values # Features
y = df_ml['Attrition'].values # Target: 0=No, 1=Yes

print('Feature shape:', X.shape)
print('Target shape:', y.shape)
print('Target classes (0=No Attrition, 1=Attrition):', np.unique(y))
```

```
Feature shape: (1470, 31)
Target shape: (1470,)
Target classes (0=No Attrition, 1=Attrition): [0 1]
```

```
# Train/Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

```
# Impute missing values (mean strategy)
imputer = SimpleImputer(strategy='mean')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)
```

```
print('Training set:', X_train_imputed.shape)
print('Test set:', X_test_imputed.shape)
```

```
Training set: (1176, 31)
Test set: (294, 31)
```

✓ KNN (Classification) — Target: Attrition (Yes / No)

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

knn_model = KNeighborsClassifier()
knn_model.fit(X_train_imputed, y_train)
y_pred_knn = knn_model.predict(X_test_imputed)

print('k-NN Accuracy:', accuracy_score(y_test, y_pred_knn))
print(classification_report(y_test, y_pred_knn, target_names=['No Attrition', 'Attrition']))
```

```
k-NN Accuracy: 0.8537414965986394
```

	precision	recall	f1-score	support
No Attrition	0.88	0.96	0.92	255
Attrition	0.36	0.13	0.19	39
accuracy			0.85	294
macro avg	0.62	0.55	0.55	294
weighted avg	0.81	0.85	0.82	294

✓ Decision Tree

```
from sklearn.tree import DecisionTreeClassifier

DT_model = DecisionTreeClassifier(random_state=42)
DT_model.fit(X_train_imputed, y_train)
y_pred_dt = DT_model.predict(X_test_imputed)

print('Decision Tree Accuracy:', accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt, target_names=['No Attrition', 'Attrition']))
```

```
Decision Tree Accuracy: 0.7585034013605442
```

	precision	recall	f1-score	support
No Attrition	0.87	0.85	0.86	255
Attrition	0.12	0.13	0.12	39

accuracy			0.76	294
macro avg	0.49	0.49	0.49	294
weighted avg	0.77	0.76	0.76	294

Naïve Bayes

```
from sklearn.naive_bayes import GaussianNB

NB_model = GaussianNB()
NB_model.fit(X_train_imputed, y_train)
y_pred_nb = NB_model.predict(X_test_imputed)

print('Naive Bayes Accuracy:', accuracy_score(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_nb, target_names=['No Attrition', 'Attrition']))
```

```
Naive Bayes Accuracy: 0.8333333333333334
      precision    recall  f1-score   support

No Attrition      0.93      0.88      0.90      255
Attrition         0.40      0.54      0.46       39

   accuracy
macro avg      0.66      0.71      0.68      294
weighted avg    0.86      0.83      0.84      294
```

Random Forest (Bagging)

```
from sklearn.ensemble import RandomForestClassifier

RF_model = RandomForestClassifier(random_state=42)
RF_model.fit(X_train_imputed, y_train)
y_pred_rf = RF_model.predict(X_test_imputed)

print('Random Forest Accuracy:', accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf, target_names=['No Attrition', 'Attrition']))
```

```
Random Forest Accuracy: 0.8707482993197279
      precision    recall  f1-score   support

No Attrition      0.88      0.99      0.93      255
Attrition         0.57      0.10      0.17       39

   accuracy
macro avg      0.72      0.55      0.55      294
weighted avg    0.84      0.87      0.83      294
```

AdaBoost

```
from sklearn.ensemble import AdaBoostClassifier

ADA_model = AdaBoostClassifier(random_state=42)
ADA_model.fit(X_train_imputed, y_train)
y_pred_ada = ADA_model.predict(X_test_imputed)
```